DBMS – SQL QUERY IMPLEMENTATION FOR E-COMMERCE DATABASE

Objectives

1. Perform SELECT, WHERE, ORDER BY, and DISTINCT queries.
2. Search products based on price, category, and availability.
3. Retrieve customer and product information.
4. Apply filtering conditions.
5. Generate basic business reports.

1. Basic SQL Queries

SELECT Query

The SELECT statement is used to retrieve data from a table.

Display all products

```
SQL> SELECT * FROM Product;

PRODUCT_ID PRODUCT_NAME    BRAND_ID CATEGORY_ID SIZE COLOR  PRICE STOCK DESCRIPTION
---------- -------------- -------- ----------- ---- ------ ----- ----- -----------
       101 Cotton T-Shirt         4           1 M    Blue     799    60 Daily wear
       102 Running Shoes          1           2 9    Black   4999    25 Sports shoes
       103 Kurta Set              3           3 L    Maroon  1899    35 Festive wear
       104 Track Pants            2           4 M    Grey    1499     0 Gym wear
       105 Casual Sneakers        2           2 8    White   3299    18 Daily wear
       106 Printed Top            4           1 S    Pink     599    50 Casual wear

6 rows selected.
```

Display selected product details

```
SQL> SELECT Product_ID, Product_Name, Price FROM Product;

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       101 Cotton T-Shirt    799
       102 Running Shoes    4999
       103 Kurta Set        1899
       104 Track Pants      1499
       105 Casual Sneakers  3299
       106 Printed Top       599

6 rows selected.
```

WHERE Clause

The WHERE clause is used to filter records based on a condition.

Find products with price less than 1000

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < 1000;

PRODUCT_ID PRODUCT_NAME   PRICE
---------- ------------- -----
       101 Cotton T-Shirt   799
       106 Printed Top      599

2 rows selected.
```

Find products with price greater than 1500

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > 1500;

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       102 Running Shoes    4999
       103 Kurta Set        1899
       105 Casual Sneakers  3299

3 rows selected.
```

ORDER BY

The ORDER BY clause is used to sort the query results.

Products from lowest to highest price

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  ORDER BY Price ASC;

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       106 Printed Top       599
       101 Cotton T-Shirt    799
       104 Track Pants      1499
       103 Kurta Set        1899
       105 Casual Sneakers  3299
       102 Running Shoes    4999

6 rows selected.
```

Products from highest to lowest price

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       102 Running Shoes    4999
       105 Casual Sneakers  3299
       103 Kurta Set        1899
       104 Track Pants      1499
       101 Cotton T-Shirt    799
       106 Printed Top       599

6 rows selected.
```

DISTINCT

The DISTINCT keyword is used to display unique values.

Display unique categories used by products

```
SQL> SELECT DISTINCT Category_ID
  2  FROM Product
  3  ORDER BY Category_ID;

CATEGORY_ID
-----------
          1
          2
          3
          4

4 rows selected.
```

Display unique brands used by products

```
SQL> SELECT DISTINCT Brand_ID
  2  FROM Product
  3  ORDER BY Brand_ID;

BRAND_ID
--------
       1
       2
       3
       4

4 rows selected.
```

2. Search Products

Search Products Based on Price

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price BETWEEN 500 AND 2000;

PRODUCT_ID PRODUCT_NAME   PRICE
---------- ------------- -----
       101 Cotton T-Shirt   799
       103 Kurta Set       1899
       104 Track Pants     1499
       106 Printed Top      599

4 rows selected.
```

This displays products whose price is between Rs. 500 and Rs. 2000.

Search Products Based on Category

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE Category_ID = 1;

PRODUCT_ID PRODUCT_NAME   CATEGORY_ID
---------- ------------- -----------
       101 Cotton T-Shirt           1
       106 Printed Top              1

2 rows selected.
```

This displays products belonging to category 1 (Topwear).

Search Products Based on Availability

Products currently in stock

```
SQL> SELECT Product_ID, Product_Name, Stock
  2  FROM Product
  3  WHERE Stock > 0;

PRODUCT_ID PRODUCT_NAME    STOCK
---------- -------------- -----
       101 Cotton T-Shirt     60
       102 Running Shoes      25
       103 Kurta Set          35
       105 Casual Sneakers    18
       106 Printed Top        50

5 rows selected.
```

Products that are out of stock

```
SQL> SELECT Product_ID, Product_Name, Stock
  2  FROM Product
  3  WHERE Stock = 0;

PRODUCT_ID PRODUCT_NAME STOCK
---------- ----------- -----
       104 Track Pants      0

1 row selected.
```

3. Retrieve Customer and Product Information

Display all customers

```
SQL> SELECT Customer_ID, First_Name, Last_Name, Email, Phone_Number
  2  FROM Customer;

CUSTOMER_ID FIRST_NAME LAST_NAME EMAIL             PHONE_NUMBER
----------- ---------- --------- ----------------- ------------
        101 Anitha     Kumar     anitha@gmail.com  9876501234
        102 Rahul      Sharma    rahul@gmail.com   9876505678
        103 Priya      Nair      priya@gmail.com   9876509012
        104 Karthik    Raja      karthik@gmail.com 9876503456
        105 Sneha      Iyer      sneha@gmail.com   9876507890

5 rows selected.
```

Search for a particular customer

```
SQL> SELECT Customer_ID, First_Name, Last_Name, Email, Phone_Number, Gender, Address
  2  FROM Customer
  3  WHERE Customer_ID = 101;

CUSTOMER_ID FIRST_NAME LAST_NAME EMAIL           PHONE_NUMBER GENDER ADDRESS
----------- ---------- --------- --------------- ------------ ------ -------
        101 Anitha     Kumar     anitha@gmail.com 9876501234   F      Chennai

1 row selected.
```

Display product details with brand and category names

```
SQL> SELECT p.Product_ID, p.Product_Name, b.Brand_Name, c.Category_Name, p.Price
  2  FROM Product p
  3  JOIN Brand b
  4      ON p.Brand_ID = b.Brand_ID
  5  JOIN Category c
  6      ON p.Category_ID = c.Category_ID;

PRODUCT_ID PRODUCT_NAME    BRAND_NAME CATEGORY_NAME PRICE
---------- --------------- ---------- ------------ -----
       101 Cotton T-Shirt  Roadster   Topwear        799
       102 Running Shoes   Nike       Footwear      4999
       103 Kurta Set       Biba       Ethnic Wear   1899
       104 Track Pants     Puma       Sportswear    1499
       105 Casual Sneakers Puma       Footwear      3299
       106 Printed Top     Roadster   Topwear        599

6 rows selected.
```

4. Apply Filtering Conditions

Multiple conditions can be combined using AND, OR, and NOT.

AND Condition

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < 2000
  4  AND Category_ID = 1;

PRODUCT_ID PRODUCT_NAME   PRICE
---------- ------------- -----
       101 Cotton T-Shirt   799
       106 Printed Top      599

2 rows selected.
```

This displays products that satisfy both conditions.

OR Condition

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Category_ID = 1
  4  OR Category_ID = 3;

PRODUCT_ID PRODUCT_NAME   PRICE
---------- ------------- -----
       101 Cotton T-Shirt   799
       103 Kurta Set       1899
       106 Printed Top      599

3 rows selected.
```

This displays products belonging to either category 1 or category 3.

NOT Condition

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE NOT Category_ID = 1;

PRODUCT_ID PRODUCT_NAME    CATEGORY_ID
---------- -------------- -----------
       102 Running Shoes             2
       103 Kurta Set                 3
       104 Track Pants               4
       105 Casual Sneakers           2

4 rows selected.
```

This displays all products that do not belong to category 1.

IN Operator

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Category_ID IN (2, 4);

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       102 Running Shoes    4999
       104 Track Pants      1499
       105 Casual Sneakers  3299

3 rows selected.
```

BETWEEN Operator

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price BETWEEN 1000 AND 3500;

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       103 Kurta Set        1899
       104 Track Pants      1499
       105 Casual Sneakers  3299

3 rows selected.
```

LIKE Operator

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Product_Name LIKE 'C%';

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       101 Cotton T-Shirt    799
       105 Casual Sneakers  3299

2 rows selected.
```

This displays products whose name starts with the letter C.

5. Basic Business Reports

Product Price Report

```
SQL> SELECT
  2      Product_ID,
  3      Product_Name,
  4      Price
  5  FROM Product
  6  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME    PRICE
---------- -------------- -----
       102 Running Shoes    4999
       105 Casual Sneakers  3299
       103 Kurta Set        1899
       104 Track Pants      1499
       101 Cotton T-Shirt    799
       106 Printed Top       599

6 rows selected.
```

Customer Order Report

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      o.Order_Status,
  7      o.Total_Amount
  8  FROM Customer c
  9  JOIN Orders o
 10      ON c.Customer_ID = o.Customer_ID
 11  ORDER BY o.Order_ID DESC;

CUSTOMER_ID CUSTOMER_NAME ORDER_ID ORDER_DATE ORDER_STATUS TOTAL_AMOUNT
----------- ------------ -------- ---------- ------------ ------------
        104 Karthik Raja       205 20-OCT-26  Processing           2397
        101 Anitha Kumar       204 18-OCT-26  Processing           3299
        103 Priya Nair         203 15-OCT-26  Shipped              2698
        102 Rahul Sharma       202 12-OCT-26  Delivered            4999
        101 Anitha Kumar       201 10-OCT-26  Delivered            2197

5 rows selected.
```

Product Order Report

```
SQL> SELECT
  2      oi.Order_ID,
  3      oi.Product_ID,
  4      oi.Quantity,
  5      oi.Unit_Price,
  6      oi.Quantity * oi.Unit_Price AS Subtotal
  7  FROM Order_Item oi
  8  ORDER BY Subtotal DESC;

ORDER_ID PRODUCT_ID QUANTITY UNIT_PRICE SUBTOTAL
-------- ---------- -------- ---------- --------
     202        102        1       4999     4999
     204        105        1       3299     3299
     205        101        3        799     2397
     203        103        1       1899     1899
     201        101        2        799     1598
     203        101        1        799      799
     201        106        1        599      599

7 rows selected.
```

Customer-wise Total Orders

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      COUNT(o.Order_ID) AS Total_Orders
  5  FROM Customer c
  6  JOIN Orders o
  7      ON c.Customer_ID = o.Customer_ID
  8  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  9  ORDER BY Total_Orders DESC, c.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME TOTAL_ORDERS
----------- ------------ ------------
        101 Anitha Kumar             2
        102 Rahul Sharma             1
        103 Priya Nair               1
        104 Karthik Raja             1

4 rows selected.
```

Customer-wise Total Purchase Amount

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      SUM(o.Total_Amount) AS Total_Purchase
  5  FROM Customer c
  6  JOIN Orders o
  7      ON c.Customer_ID = o.Customer_ID
  8  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  9  ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME TOTAL_PURCHASE
----------- ------------ --------------
        101 Anitha Kumar            5496
        102 Rahul Sharma            4999
        103 Priya Nair              2698
        104 Karthik Raja            2397

4 rows selected.
```

Product-wise Quantity Sold

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(oi.Quantity) AS Total_Sold
  5  FROM Product p
  6  JOIN Order_Item oi
  7      ON p.Product_ID = oi.Product_ID
  8  GROUP BY p.Product_ID, p.Product_Name
  9  ORDER BY Total_Sold DESC, p.Product_ID;

PRODUCT_ID PRODUCT_NAME    TOTAL_SOLD
---------- -------------- ----------
       101 Cotton T-Shirt          6
       102 Running Shoes           1
       103 Kurta Set               1
       105 Casual Sneakers         1
       106 Printed Top             1

5 rows selected.
```

Low Stock Report

```
SQL> SELECT Product_ID, Product_Name, Stock
  2  FROM Product
  3  WHERE Stock < 20
  4  ORDER BY Stock ASC;

PRODUCT_ID PRODUCT_NAME    STOCK
---------- -------------- -----
       104 Track Pants         0
       105 Casual Sneakers    18

2 rows selected.
```